

## Final Resilience Design Guidelines

Chain: Client → MS1 → MS2 → MS3 (Istio) → MS4 (no sidecar) → Core App (no sidecar)

---

### 1. Foundational Design Rules

1. **Retry ownership = the hop closest to the unprotected/unreliable dependency.** In this topology, that's MS4 → Core App. Not MS3, not MS1/MS2.
  2. **A hop only retries its own failure, never a failure that's already been retried and exhausted further downstream.**
  3. **Idempotency is a prerequisite for retry, not optional.** No idempotency guarantee → no automatic retry, regardless of how "transient" the error looks.
  4. **Status codes must be translated at every hop that has real logic (MS4), not blindly proxied.** Codes are a contract with the caller, not a passthrough of what you happened to receive.
  5. **Infra (Istio) handles connection-level resilience declaratively; application code handles business-aware resilience (idempotency, fallback, Core-App-specific circuit breaking).** Don't duplicate one in the other.
  6. **Timeouts must nest: each upstream hop's timeout must comfortably exceed everything happening beneath it, including retries.**
  7. **Every retry/circuit-breaker/fallback decision should be observable** — traced, logged, and metriced — or it's undebuggable in production.
- 

### 2. Full HTTP Status Code Reference for This Chain

#### 2xx — no action needed

| Code        | Meaning                        |
|-------------|--------------------------------|
| 200/201/204 | Success. No resilience action. |

#### 3xx — redirects

Generally not part of service-to-service resilience design; ensure clients follow or reject per API contract. Not a retry trigger.

#### 4xx — client/request errors: NEVER retry automatically

| Code                     | Meaning                              | Action                                                                                                                  |
|--------------------------|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| 400 Bad Request          | Malformed request                    | Propagate as-is, no retry, log for debugging (likely a caller bug)                                                      |
| 401 Unauthorized         | Auth failure                         | No retry (unless token refresh + single retry is explicitly designed)                                                   |
| 403 Forbidden            | Authz failure                        | No retry                                                                                                                |
| 404 Not Found            | Resource doesn't exist               | No retry                                                                                                                |
| 408 Request Timeout      | Client took too long sending request | Caller-side issue; safe to retry only if idempotent, but rare in service-to-service                                     |
| 409 Conflict             | Business rule / concurrency conflict | No retry — needs conflict resolution logic, not blind retry                                                             |
| 422 Unprocessable Entity | Semantically invalid payload         | No retry                                                                                                                |
| 429 Too Many Requests    | Rate limited                         | <b>Retry, but only respecting Retry-After</b> — never immediately; treat as a signal to slow down, not a transient blip |

**Industry standard:** 4xx = deterministic client error. Retrying without changing the request is universally discouraged (AWS, Google SRE, Microsoft all treat this identically) — it wastes resources and can trip abuse/rate-limit detection.

#### 5xx — server-side errors: conditional retry, and codes should be deliberately chosen, not default

| Code                      | Meaning in this chain                         | Who should retry?                                                |
|---------------------------|-----------------------------------------------|------------------------------------------------------------------|
| 500 Internal Server Error | The responding service's own code/logic broke | Ambiguous — could be transient crash or deterministic bug. Retry |

| Code                    | Meaning in this chain                                                                                            | Who should retry?                                                                                                                                          |
|-------------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                         |                                                                                                                  | cautiously (max 1) or not at all without more signal. <b>Should NOT be used to mean "my downstream failed"</b> — reserve for genuine own-service failures. |
| 502 Bad Gateway         | MS4 called Core App and got an invalid/broken response (already retried internally by MS4 if idempotent)         | MS4 already retried before emitting this. <b>MS3 should NOT retry.</b>                                                                                     |
| 503 Service Unavailable | MS4 is deliberately refusing — circuit breaker open, overloaded, bulkhead full, draining for deploy              | <b>MS3 should NOT retry immediately</b> — respect <code>Retry-After</code> . Often means Core App wasn't even called this time.                            |
| 504 Gateway Timeout     | MS4 called Core App and it didn't respond within MS4's timeout (already retried internally by MS4 if idempotent) | MS4 already retried before emitting this. <b>MS3 should NOT retry.</b>                                                                                     |

#### Industry standard alignment:

- RFC 9110 (HTTP semantics) explicitly defines 502/504 as "this server, acting as a gateway, got a bad/no response from the server it accessed" — i.e., the failure is attributed one hop further down, exactly the semantic you're relying on.
- Google SRE Workbook and AWS Builders' Library both recommend: **use 503 for deliberate load-shedding/circuit-breaker-open states, with `Retry-After`**, and reserve 502/504 for genuine upstream failures — this matches your design exactly.

#### Connection-level failures (no HTTP status code at all)

| Failure            | Meaning                       | Retryable?                                      |
|--------------------|-------------------------------|-------------------------------------------------|
| Connection timeout | Couldn't establish connection | Yes — fresh, hop-local, safe to let Istio retry |
| Connection refused | Target not listening          | Yes — fresh, hop-local                          |

| Failure                | Meaning                           | Retryable?                                                                                                                |
|------------------------|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Connection reset       | Dropped mid-request               | Yes, but if it was a write, treat as <b>unknown outcome</b> — needs idempotency key                                       |
| Read/response timeout  | Sent request, no response in time | <b>Unknown outcome</b> — the receiving side may have processed it. Never blindly retry non-idempotent operations on this. |
| DNS resolution failure | Can't resolve hostname            | Yes — infra issue, hop-local                                                                                              |
| TLS handshake failure  | Cert/protocol mismatch            | Usually not retry-worthy — indicates misconfiguration, retrying won't fix it                                              |

### 3. Layer-by-Layer Responsibility (Your Specific Topology)

| Hop                | Mesh protection?           | Resilience owner                                                               | What's needed                                                                                                                                                                    |
|--------------------|----------------------------|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Client<br>→<br>MS1 | No                         | MS1 app / client                                                               | Client-driven retry only on explicit user action; not automatic                                                                                                                  |
| MS1<br>→<br>MS2    | No                         | MS1 app                                                                        | Propagate/fallback only — don't re-retry MS3/MS4/Core App failures                                                                                                               |
| MS2<br>→<br>MS3    | No                         | MS2 app                                                                        | Propagate/fallback only                                                                                                                                                          |
| MS3<br>→<br>MS4    | <b>Yes — Istio sidecar</b> | Istio config<br>( <code>VirtualService</code> / <code>DestinationRule</code> ) | <code>retryOn: connect-failure,refused-stream,reset</code> — explicitly exclude <code>gateway-error /502/503/504</code> .<br>Outlier detection for pod health. Sensible timeout. |

| Hop                  | Mesh protection?            | Resilience owner     | What's needed                                                                                                                               |
|----------------------|-----------------------------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| MS4<br>→<br>Core App | <b>No protection at all</b> | MS4 application code | Full stack: retry+backoff+jitter, idempotency keys for writes, circuit breaker (business-aware), timeout, fallback, status-code translation |

**This is the single most important structural takeaway: nearly all your custom resilience code effort should go into MS4's Core App client wrapper — everything else is either mesh config or simple propagation.**

#### 4. MS4 → Core App: Required Components (the one hop needing full build-out)

1. **Idempotency key generation/passthrough** for any write operation (POST/non-idempotent PUT) — generated by the originating caller if possible, or by MS4, and honored by Core App to detect duplicate processing.
2. **Bounded retry with exponential backoff + jitter** (2–3 attempts) — only for idempotent operations or operations with a valid idempotency key.
3. **Circuit breaker** (Resilience4j / Polly / equivalent) around the Core App client — trips on error-rate threshold, not single failures; must count connection-level failures, not just HTTP 5xx.
4. **Bulkhead** — isolated connection/thread pool for Core App calls so Core App slowness doesn't starve MS4's other work.
5. **Timeout** shorter than whatever MS3 imposes on MS4, leaving room for MS4's own retry attempts to complete within budget.
6. **Status code translation layer** — never proxy Core App's raw response; always emit MS4's own deliberate 500/502/503/504 per the table above.
7. **Fallback** — cached/default/degraded response where business-viable, so MS3 doesn't always see a hard failure.
8. **Structured error metadata in the body** (for humans/tracing, not for Istio) — `origin, retryable, circuitBreakerState, correlationId, retryAfterMs`.

## 5. MS3 (Istio) Configuration Principles

- **Explicit `retryOn` , never the `gateway-error` shorthand** in this design — it bundles 502/503/504 together, all of which mean "don't retry" here.
  - **`outlierDetection`** on the `DestinationRule` for MS4, to eject genuinely unhealthy MS4 pods — purely infra-level, no app-awareness needed.
  - **Timeout on `VirtualService`** for the MS3 → MS4 call, sized to comfortably exceed MS4's internal Core App retry budget.
  - **Do not attempt to inspect/act on response bodies at the sidecar** (would require custom `EnvoyFilter` /WASM) — keep the mesh dumb and fast; business-aware decisions stay in MS4's app code.
- 

## 6. Circuit Breaker — Industry-Standard Parameters (Netflix Hystrix / Resilience4j conventions)

| Parameter                    | Typical industry default                         | Notes                                                                                                                                                |
|------------------------------|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Failure rate threshold       | 50%                                              | Trip when half of recent calls fail                                                                                                                  |
| Sliding window size          | 10–20 calls (count-based) or 10–60s (time-based) | Count-based preferred for low-traffic services                                                                                                       |
| Wait duration in open state  | 30–60s                                           | Cooldown before allowing test traffic                                                                                                                |
| Permitted calls in half-open | 3–5                                              | Small trickle to test recovery                                                                                                                       |
| Slow-call duration threshold | Often added alongside failure rate               | A call that's "successful but too slow" should also count toward tripping — pure error-rate-only breakers miss degraded-but-not-failing dependencies |

---

## 7. Retry — Industry-Standard Parameters (AWS, Google SRE, Azure Architecture Center)

- **Max attempts: 2–3.** AWS SDKs commonly default to 3; Google SRE Workbook explicitly warns against unbounded or high retry counts due to amplification risk.
  - **Exponential backoff with full jitter** — AWS's own "Exponential Backoff and Jitter" architecture blog is the widely cited reference; full jitter (random between 0 and computed max) outperforms fixed or partial jitter in preventing thundering herd.
  - **Retry budget / rate limiting on retries themselves** — Google SRE recommends capping retries as a percentage of total request volume system-wide (e.g., no more than 10%), so retries can't themselves become a load-driving incident during a partial outage.
  - **Respect Retry-After** whenever present (429, 503) — explicit signal beats computed backoff.
- 

## 8. Observability — Non-Negotiable Regardless of Scale

- **Distributed tracing (OpenTelemetry)** across every hop, including Istio sidecars — not just app-to-app spans.
  - **Correlation ID** propagated end-to-end from Client through to Core App and back.
  - **Metrics per hop:** retry count, retry success rate, circuit breaker state transitions (closed/open/half-open), p50/p95/p99 latency, error rate by status code category (4xx vs 5xx vs connection-level).
  - **Alert on the right signal:** circuit breaker open state and sustained 5xx rate are actionable; individual retries are not (too noisy to page on).
  - **Separate dashboards/alerts for "MS4 unreachable" (infra/network issue) vs "MS4 reachable but Core App failing" (dependency issue)** — they route to different teams.
- 

## 9. Idempotency — Industry Pattern (Stripe, AWS, Google-style idempotency keys)

- Caller (MS3 or MS4, depending on where the write originates) generates a unique idempotency key per logical operation (UUID).
- Key is passed through headers (e.g., `Idempotency-Key`) to Core App.
- Core App (or MS4, if it fronts this logic) stores recently-seen keys with their result for a bounded window (e.g., 24h) and returns the original result on a

duplicate key, instead of reprocessing.

- This makes retry safe **regardless of whether the original request actually reached/completed at Core App** — it removes the "unknown outcome" ambiguity entirely for that operation, which is the single biggest practical risk in this whole design.

---

## 10. Anti-Patterns to Explicitly Avoid

1. **Using `gateway-error` shorthand in Istio when your own services distinguish 502/503/504 semantically** — causes Istio to retry things you've deliberately marked "already exhausted."
2. **Retrying at every hop independently** — causes multiplicative amplification ( $N^n$  actual calls to Core App from one failure).
3. **Blindly proxying raw downstream status codes upstream** — destroys the ability of callers to make correct decisions.
4. **Retrying non-idempotent writes without an idempotency key** — root cause of duplicate-charge/duplicate-order incidents industry-wide.
5. **Circuit breaker based on error rate alone, ignoring slow/degraded calls** — a dependency that's "succeeding" at 10x normal latency should still trip protective behavior.
6. **Uniform resilience rigor across all calls** — over-engineering low-risk reads the same as high-risk writes wastes effort and adds unnecessary failure surface (more config = more to misconfigure).
7. **No `Retry-After` handling** — computed backoff ignoring an explicit server hint leads to retrying sooner than the server told you to, worsening the exact problem you're retrying around.
8. **Health checks encoding business logic** — keep liveness/readiness dumb ("can this process serve traffic"), keep circuit breakers business-aware separately.

---

## 11. One-Page Summary Table

| Concern                                | Where it lives                                                                    | Standard reference |
|----------------------------------------|-----------------------------------------------------------------------------------|--------------------|
| Connection-level retry, LB, pod health | Istio<br>( <code>VirtualService</code> / <code>DestinationRule</code> )<br>at MS3 | Istio/Envoy docs   |

| Concern                        | Where it lives                       | Standard reference                      |
|--------------------------------|--------------------------------------|-----------------------------------------|
| Business-aware retry + backoff | MS4 app code (Core App client only)  | AWS Backoff+Jitter, Google SRE Workbook |
| Idempotency                    | MS4/Core App, Idempotency-Key header | Stripe/AWS idempotency pattern          |
| Circuit breaker                | MS4 app code (Resilience4j/Polly)    | Netflix Hystrix-derived conventions     |
| Status code semantics          | MS4 (translates before responding)   | RFC 9110                                |
| Rate limiting compliance       | Any hop receiving 429                | Retry-After header, RFC 9110            |
| Observability                  | All hops                             | OpenTelemetry                           |
| Timeout nesting                | All hops, decreasing downstream      | Azure Architecture Center, Google SRE   |

---

**Design summary for your exact topology:** Istio at MS3 handles the *only* mesh-protected hop with pure config. Every other hop is unprotected infra-wise, but only MS4 → Core App needs full custom resilience engineering, because it's the hop touching the actual unreliable dependency furthest from the client. MS1, MS2 stay simple — propagate or fallback, never re-retry. This concentrates complexity in exactly one place instead of spreading it thin and inconsistently across the whole chain.